P3669R3
Non-Blocking Support for `std::execution`

Published Proposal, 2025-10-06

This version:
http://wg21.link/P3669R3

Author:
Detlef Vollmann

Audience:
SG1, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
gitlab.com/cppzs/papers/-/blob/main/source/P3669R3.bs

Abstract

`std::execution` as currently specified doesn't provide support for non-blocking operations. This proposal tries to fix this.

Table of Contents

§ 1. Revision History

P3669R3 revises P3669R2 - 2025-06-19 as follows:

- added another example
- added link to implementations
- wording fixes
- added naming discussion
- removed modifications to P0260

P3669R2 revises P3669R1 - 2025-05-17 as follows:

- added more rationale
- renamed `concurrent_scheduler` to `try_scheduler` as a placeholder based on SG1 feedback
- added usage example

P3669R1 revises P3669R0 - 2025-04-14 as follows:

- drops *concurrent-op-state* and introduces `try_schedule`

§ 2. Introduction

Some execution environments want to make sure that specific operations are non-blocking. These can not signal an event using the facilities of `std::execution` as currently specified as it doesn't provide operations that are guaranteed to be non-blocking.

E.g. this applies to signal handlers for asynchronous signals. A framework that claims to support concurrent and asynchronous execution but doesn't allow input from asynchronous signal handlers is just broken.

This became apparent in the proposal for concurrent queues *C++ Concurrent Queues (P0260)*, which has support for non-blocking usage, but the problem is far bigger than the concurrent queue issue. There are many events that are notified by some kind of signal (e.g. in embedded systems essentially all incoming data is notified using an interrupt, which is a signal in the C++ sense) that you want to consume using S/R mechanisms.

The non-blocking operations of concurrent queues got a very weird interface when interfacing with `std::execution` due to the lack of non-blocking signalling to an async waiter. This interface can be fixed as soon as there exist a signal-safe mechanism to notify S/R.

This proposal is completely independent from *C++ Concurrent Queues (P0260)* as the problem of signalling an event in a non-blocking way is not specific to concurrent queues.

§ 3. Design

In `std::execution` the basic operation to signal an event is to schedule a continuation. In practice this generally means that a `start` operation is called on an operation state that comes from a sender provided by a scheduler.

The initial revision of this paper proposed to add a `bool try_start()` operation to the operation states that come from a scheduler and require it to be non-blocking (and to return `false` if it would block).

After feedback from several people this initial design was dropped. Instead a new CPO `try_schedule()` is proposed.

If a `start()` on the operation state of a sender returned by `try_schedule()` would block (e.g. for inserting the operation into a work queue protected by a mutex), it immediately calls `set_error(would_block_t())` on the connected receiver.

Unfortunately it is not possible for all kind of schedulers to provide `try_schedule()`. Schedulers that are not backed by an execution context that maintains a work queue but run the operation right away are an example. Schedulers that enqueue the work on a different system are another example.

For this reason, this paper proposes a new concept `try_scheduler`. `try_scheduler` requires the `try_schedule` operation.

4. Naming

Originally, this paper proposed a `try_start` function as member of a special operation state and the related concept was called `concurrent-op-state`.

After changing to a new CPO named `try_schedule` the related concept was called `concurrent_scheduler`.

`concurrent_scheduler` wasn't liked by SG1 in Sofia and the concept was renamed to `try_scheduler`.

After asking on the SG1 and LEWG reflectors, other names were proposed:

- `can_schedule` (and then `can_scheduler`)
- `non_blocking_schedule` and `non_blocking_scheduler`
- keeping `try_schedule` for the CPO but renaming the concept to `non_blocking_scheduler`

`try_schedule` meaning non-blocking (and therefore potentially failing) would be consistent with `try_lock`, `try_push` and `try_pop`, but not with `try_emplace`.

This paper keeps `try_schedule` and `try_scheduler`, but it's up to SG1 and LEWG to decide on the final name.

5. Examples and Implementation

5.1. Implementation

An implementation of this proposal on top of execution, stdexec and ustdex is available at https://git-lab.com/cppzs/std-execution-examples/externals.

§ 5.2. Timer

A simple timer example based on POSIX SIGALRM. This example works without a special execution context. This code is simplified and doesn't compile. The full code can be found at std-execution-examples/timer.

```
class AsyncSleep
{
public:
    template <typename Dur>
    AsyncSleep(Dur time_)
      : time(std::chrono::duration_cast<std::chrono::microseconds>(time_))
    {}

    template <class Recv>
    auto connect(Recv &&r) -> AlarmOp<std::decay_t<Recv>>
    {
        return {std::forward<Recv>(r), this};
    }

private:
    // helper receiver to catch the would_block_t
    template <execution::receiver InnerRecv>
    struct TryReceiver
    {
        using receiver_concept = execution::receiver_t;

        TryReceiver(InnerRecv *r_, AlarmOp<InnerRecv> *op_)
          : op(op_)
          , r(r_)
        {}

        void set_error(execution::would_block_t) noexcept
        {
            op->valid = false; // set the flag to fail
        }

        // all other completions are simply forwarded to the stored receive

        AlarmOp<InnerRecv> *op;
        InnerRecv *r;
    };
```

```
template <execution::receiver Recv>
struct AlarmOp
{
    AlarmOp(Recv &&r_, AsyncSleep *as)
      : time(as->time)
      , r(std::forward<Recv>(r_))
      , valid(true)
      , sch(execution::get_scheduler(execution::get_env(r)))
      , tryContOp(getTryContOp<Recv>())
    {}

    // here we get the operation state from the scheduler
    template <execution::receiver R>
    requires HasTryScheduler<R>
    constexpr auto getTryContOp() noexcept
    {
        return execution::connect(sch.try_schedule(),
                                  TryReceiver<Recv>(&r, this));
    }

    // this is called by the signal handle and must be signal-safe
    bool scheduleContinuation()
    {
        tryContOp.start();      // start on try scheduler

        if (not valid)    // scheduling failed
        {
            // reset helper op; we keep this operation
            tryContOp.~TryContOpT();
            new (&tryContOp) TryContOpT(getTryContOp<Recv>());
            valid = true;

            return false;
        }

        // we could successfully schedule the continuation
        // the continuation will call set_value on the receiver
        curOp = nullptr;
        return true;
    }

    // start the timer
```

```
        void start() noexcept
        {
            curOp = this;
            setHandler();
            setTimer(time);
        }

        void doSignal()  // this is called by the signal handler
        {
            if (scheduleContinuation())
            {
                curOp = this;
            }
            else
            {
                // retry a bit later
                setHandler();
                setTimer(1us);
            }
        }

        Recv r;
        bool valid;
        SchedT sch;
        TryContOpT tryContOp;
    };
};

int main()
{
    execution::sender auto s = AsyncSleep(20ms) | ex::then([] { /*...*/ });
}
```

§ 5.3. `bounded_queue`

Here's an extract from the implementation of `bounded_queue` that shows how `try_schedule` is used there.

`RetrieveOp` represents the operation of an `async_pop`. `scheduleTryContinuation` is called from `try_push` if the queue is currently empty.

```
// helper class to intercept the would_block from try_schedule
template <execution::receiver InnerRecv>
struct PopContReceiver
{
  void set_error(execution::would_block_t) noexcept
  {
    op->valid = false;
  }

  // all other completions are simply forwarded to r

  InnerRecv *r;
  RetrieveOpBase *op;
};

template <execution::receiver Recv>
struct RetrieveOp : RetrieveOpBase
{
  TryContOpT tryContOp;
  Recv r;
  bounded_queue *q;

  template <execution::receiver Rc>
  RetrieveOp(Rc &&r_, bounded_queue *q_)
    : r(std::forward<Rc>(r_))
    , tryContOp(getTryContOp<Recv>(this))
    , q(q_)
  {}

  // this is a helper to get a continuation op with a try scheduler
  template <execution::receiver R>
    requires HasTryScheduler<R>
  constexpr auto getTryContOp(RetrieveOpBase *self) noexcept
  {
    this->valid = true;
    auto sch = execution::get_scheduler(execution::get_env(r));
    return execution::connect(execution::try_schedule(sch),
                              PopContReceiver<Recv>(&r, self));
  }
```

```
  bool scheduleTryContinuation(std::optional<T> &&v) override
  {
    tryContOp.start();      // start on try scheduler

    if (not this->valid)    // scheduling failed
    {
      tryContOp = getTryContOp(this); // reset helper op; we keep this oper
      return false;
    }

    // we got the continuation scheduled, drop it from our list
    --(q->asyncPopWaitCnt);
    q->asyncPoppers.pop_front();

    this->val = std::move(v);  // only move the value on success
    return true;
  };
};
```

6. Proposed Wording

6.1. `try_schedule` Additions

Add to synopsis in namespace `std::execution`:

```
struct try_scheduler_t {};

struct would_block_t {};
```

6.1.1. Try Scheduler Concept

Add to **Schedulers** [exec.sched]:

1. The concept `try_scheduler` defines the requirements of a scheduler type ([async.ops]) that provides `try_schedule()`.

```
namespace std::execution {
  template<class S, receiver R>
    concept try_scheduler =
      std::derived_from<typename std::remove_cvref_t<_Scheduler>::try_s
      scheduler<S> &&
      requires (S&& s) {
        { try_schedule(std::forward<S>(s)) } -> sender;
    }
}
```

6.1.2. `execution::try_schedule`

Add to **Sender factories** [exec.factories]:

1. `try_schedule` obtains a schedule sender ([exec.async.ops]) from a scheduler to potentially start an asynchronous operation without blocking ([defns.block]).
2. The name `try_schedule` denotes a customization point object. For a subexpression `sch`, the expression `try_schedule(sch)` is expression-equivalent to `sch.try_schedule()`.
3. *Mandates:* The type of `sch.try_schedule()` satisfies `sender`.
4. If the expression `get_completion_scheduler<set_value_t>(get_env(sch.try_schedule())) == sch` is ill-formed or evaluates to `false`, the behavior of calling `try_schedule(sch)` is undefined.
5. The expression `sch.try_schedule()` has the following semantics:
    1. Let `w` be a sender object returned by the expression, and `op` be an operation state obtained from connecting `w` to a receiver `r`.
    2. *Returns:* A sender object `w` that behaves as follows:
        1. `op.start()` starts ([async.ops]) the asynchronous operation associated with `op`, if it can be achieved without blocking.
        2. If starting the asynchronous operation would block, it immediately calls `set_error(r, would_block_t())`.
    3. `op.start()` is signal-safe.

6.1.3. `run_loop`

1. *run-loop-scheduler* models `try_scheduler`.

References

Informative References

[P0260]
C++ Concurrent Queues (P0260). URL: https://wg21.link/P0260